2026-05-08-p5-documentation

Phase 5 — Full Documentation Suite — Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development or superpowers:executing-plans. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Create 6 standalone guides, add Go doc comments to every exported symbol, update all existing materials, final anti-bluff sweep, final push.

Architecture: All guides under `docs/guides/`. Each guide is self-contained. Go doc comments added inline to all exported symbols. Existing materials updated to reflect Phase 1-5 completions.

Tech Stack: Markdown, Go doc comments, bash

Spec: `docs/superpowers/specs/2026-05-08-helixcode-zero-bluff-completion-design.md`

File Structure Map

```
docs/guides/user-manual.md                    — create
docs/guides/developer-guide.md                — create
docs/guides/api-reference.md                  — create
docs/guides/deployment-guide.md               — create
docs/guides/troubleshooting.md                — create
docs/guides/challenge-authoring.md            — create
helix_code/internal/**/*.go                    — modify (add doc comments)
helix_code/cmd/**/*.go                         — modify (add doc comments)
helix_code/applications/**/*.go                — modify (add doc comments)
GAP_ANALYSIS.md                               — modify
HELIXCODE_FEATURE_GAP_ANALYSIS.md             — modify
HELIXCODE_GAP_ANALYSIS.md                     — modify
AGENTS.md                                     — modify
docs/improvements/PROGRESS.md                 — modify
docs/CONTINUATION.md                          — modify
```

Task P5-T01: User Manual

Files: Create `docs/guides/user-manual.md`

- [ ] **Step: Write the guide covering:**

HelixCode User Manual

1. Installation
- Linux (apt, yum, snap, AppImage)
- macOS (Homebrew)
- Windows (NSIS installer)
- Aurora OS / Harmony OS
- From source

2. Getting Started
- First project: `helix start`
- First AI session: `helix auto "build a todo CLI"`
- Project structure overview

3. CLI Reference
`helix` commands
- `start` — start a new AI development session
- `auto` — autonomous development
- `server` — start the HTTP server
- `generate` — one-shot code generation
- `test` — run the test suite
- `worker` — manage distributed workers
- `notify` — send notifications
- `permissions` — manage permission rules
- `worktree` — manage git worktrees
- `hooks` — manage hook-based extensibility
- `lsp` — LSP server status
- `sandbox` — sandboxed shell execution
- `/subagents` — manage subagent teams
- `/telemetry` — OTel status
- `/sessions` — session transcripts
- `/plantree` — plan trees
- `/openhands` — workspace management
- `/git_auto_commit` — auto-commit control
- `/browser` — chrome browser automation
- `/memory` — project memory management
- `/commands` — markdown slash commands
- `/skills` — skill system
- `/theme` — color theme control
- `/edit` — smart file editing

`helix-config` commands
- `show` — display current configuration
- `set` — set a configuration value
- `delete` — delete a configuration key
- `validate` — validate configuration
- `export` — export to file
- `import` — import from file

- `backup` — create backup
- `restore` — restore from backup
- `reset` — reset to defaults
- `reload` — hot-reload config
- `watch` — watch for changes
- `templates` — list/use templates
- `history` — configuration history
- `schema` — generate JSON schema

4. Terminal UI (TUI)
- Navigation: tab between panels, enter to select
- Session list: current and past sessions
- Stats panel: tokens, cost, time
- QA integration: run/view/cancel QA sessions

5. Configuration
- Server settings (address, port, timeouts)
- Database (PostgreSQL, optional)
- Redis (optional)
- Auth (JWT, sessions)
- Worker pool settings
- Task management settings
- LLM provider settings
- Logging
- Notifications
- Environment variables reference

6. LLM Providers
- OpenAI, Anthropic, Gemini, Ollama
- Azure, Bedrock, VertexAI
- Groq, Mistral, Cohere
- xAI, DeepSeek, Qwen
- OpenRouter, HuggingFace
- Llama.cpp, Replicate, Together.ai
- How to configure each
- API key setup
- Model selection

7. Workflows
- Planning: `PlanStep`
- Building: `BuildStep`
- Testing: `TestStep`
- Refactoring: `RefactorStep`

8. FAQ
- Common issues and solutions

☐ **Commit:**

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add docs/guides/user-manual.md
git commit -m "docs(P5-T01): add user manual covering
        installation, CLI, TUI, config, providers

Phase: 5  Task: P5-T01"
```

Task P5-T02: Developer Guide

Files: Create `docs/guides/developer-guide.md`

- [ ] **Write the guide covering:**

```
# HelixCode Developer Guide

## Architecture Overview
- Package dependency map
- Data flow: API → server → agent → LLM → tools
- Client architecture: REST, CLI, TUI, Desktop, Mobile, WebSocket

## Building from Source
- Prerequisites: Go 1.24, gcc, make
- `make setup-deps`, `make build`, `make logo-assets`
- Containerized build: `make container-build`
- Cross-compilation targets

## Code Organization
- `cmd/` — entry points
- `internal/` — all packages
  - `auth/` — authentication
  - `llm/` — LLM providers + litellm
  - `server/` — HTTP server
  - `tools/` — tool ecosystem
  - `worker/` — distributed workers
  - `task/` — task management
  - `workflow/` — workflow engine
  - `memory/` — memory providers
  - `notification/` — notifications
  - `mcp/` — MCP protocol
  - `config/` — configuration
  - `verifier/` — LLMsVerifier integration
  - `repomap/` — codebase mapping
  - `quality/` — quality scoring
  - `clarification/` — ambiguity resolution
  - `plugins/` — plugin system
  - `sandbox/` — sandboxed execution
  - `session/` — session management
```

```
  - `hooks/` — hook system
  - `rules/` — rules engine
  - `project/` — project management
  - `template/` — template system
  - `performance/` — performance optimization
  - `security/` — security scanning

## Adding a New LLM Provider
1. Create `internal/llm/providers/<name>/client.go`
2. Implement `Provider` interface
3. Add to `providers/registry.go`
4. Add config to `config/config.go`
5. Write unit test + integration test + challenge
6. Add to user manual provider list

## Adding a New Tool
1. Create tool file in `internal/tools/<tool>.go`
2. Implement `tools.Tool` interface
3. Register in `tools/registry.go`
4. Add slash command in `internal/commands/`
5. Write tests + challenge

## Contributing
- Conventional commits format: `type(scope): message`
- Phase: N  Task: TNN format
- Evidence links required
- Anti-bluff: every test carries runtime evidence
- Pull request process
```

☐ **Commit**

Task P5-T03: API Reference

Files: Create `docs/guides/api-reference.md`

☐ **Write the guide based on** `api/openapi.yaml`

```
# HelixCode API Reference

## Authentication
- Bearer JWT token
- Header: Authorization: Bearer <token>
- Token expiry: configurable

## REST Endpoints
```

Health
- GET /health — service health (no auth)
- GET /api/v1/health — detailed health with DB/Redis

Auth
- POST /api/v1/auth/register — user registration
- POST /api/v1/auth/login — login, returns JWT
- POST /api/v1/auth/refresh — refresh token
- POST /api/v1/auth/logout — invalidate session

LLM
- POST /api/v1/llm/generate — generate text
- POST /api/v1/llm/generate-stream — streaming generation
- GET /api/v1/llm/models — list available models
- GET /api/v1/llm/providers — list providers
- GET /api/v1/llm/providers/{id}/status — provider health

Projects
- POST /api/v1/projects — create
- GET /api/v1/projects — list
- GET /api/v1/projects/{id} — get
- DELETE /api/v1/projects/{id} — delete

Tasks
- POST /api/v1/tasks — create task
- GET /api/v1/tasks — list
- GET /api/v1/tasks/{id} — get
- PUT /api/v1/tasks/{id} — update

Workers
- GET /api/v1/workers — list
- POST /api/v1/workers — register
- DELETE /api/v1/workers/{id} — remove

QA
- POST /api/v1/qa/session — start QA session
- GET /api/v1/qa/session/{id}/status
- GET /api/v1/qa/session/{id}/report
- GET /api/v1/qa/session/{id}/screenshot/{name}
- DELETE /api/v1/qa/session/{id}

Metrics
- GET /api/v1/metrics — runtime metrics

WebSocket (MCP)
- ws://host:port/api/v1/mcp
- JSON-RPC-like message format
- Tool execution dispatch

```
## Error Codes
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 429 Rate Limited
- 500 Internal Server Error

## Rate Limiting
- Configurable per-route limits
- Headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-
        Reset
```

- [ ] **Commit**

Task P5-T04: Deployment Guide

Files: Create `docs/guides/deployment-guide.md`

- [ ] **Write covering:**

```
# HelixCode Deployment Guide

## Docker Compose (Recommended)
- docker-compose.yml: production stack
- docker-compose-simple.yml: minimal dev
- docker-compose.test.yml: test infrastructure
- docker-compose.full-test.yml: full test infrastructure

## Kubernetes
- Helm chart (if exists)
- Manifests: deployment, service, configmap, secrets
- Persistent volumes for DB + Redis

## Environment Variables
Complete reference of all HELIX_* variables

## SSL/TLS
- Nginx reverse proxy configuration
- Let's Encrypt integration
- Self-signed certs (dev only)

## Monitoring
- Prometheus metrics endpoint
- Grafana dashboards
- Health check endpoint
```

```
- Alerting rules

## Backup & Restore
- PostgreSQL backup: pg_dump
- Redis backup: BGSAVE
- Config backup: helix-config backup

## Scaling
- Horizontal scaling with multiple server instances
- Redis session store for shared state
- Database connection pooling
- Worker pool sizing

## Security
- JWT secret rotation
- API key management (CONST-042)
- Firewall rules
- Audit logging
```

- [ ] **Commit**

Task P5-T05: Troubleshooting Guide

Files: Create `docs/guides/troubleshooting.md`

- [ ] **Write covering:**

```
# HelixCode Troubleshooting Guide

## Common Errors

### "Failed to connect to PostgreSQL"
- Check database.host in config
- Verify PostgreSQL is running
- Check credentials in HELIX_DATABASE_PASSWORD

### "Redis connection refused"
- Set redis.enabled: false to disable
- Check HELIX_REDIS_PASSWORD

### "LLM provider timeout"
- Increase llm.timeout
- Check provider status endpoint
- Fall back to another provider

### "Permission denied (publickey)" (SSH workers)
```

```
- Verify SSH key setup
- Check worker host reachability
- Run: ssh -T user@host

### "Build fails with logo-assets error"
- Run: make logo-assets

## Debug Logging
- Set logging.level: debug
- HELIX_LOG_LEVEL=debug

## Health Check
- GET /health — quick check
- GET /api/v1/health — detailed with DB/Redis status

## Performance Profiling
- pprof endpoints (if enabled)
- metrics endpoint
- Go runtime metrics

## Diagnostic Commands
- helix-config validate — check configuration
- helix worker status — check workers
- /sandbox test — test sandbox capability
- /lsp status — check LSP servers
```

- [ ] **Commit**

Task P5-T06: Challenge Authoring Guide

Files: Create `docs/guides/challenge-authoring.md`

- [ ] **Write covering:**

```
# HelixCode Challenge Authoring Guide

## Challenge Structure
```

challenges// challenge.go — main entry point (runnable with `go run`) expected.json — expected results and evidence assertions README.md — challenge description (optional)

```
## challenge.go Template
```go
package main

import (
    "crypto/sha256"
```
```

```
    "encoding/hex"
    "fmt"
    "os"
)

func main() {
    // 1. Execute the feature being tested
    // 2. Collect runtime evidence
    // 3. Verify against expected.json assertions
    // 4. Exit 0 on PASS, non-zero on FAIL

    passed := true

    // ... test logic ...

    if passed {
        fmt.Println("PASS")
        os.Exit(0)
    }
    fmt.Println("FAIL")
    os.Exit(1)
}
```

expected.json Format

```
{
  "expected_result": "PASS",
  "evidence_type": "sha256|runtime_output|compilation",
  "assertions": {
    "compilation_pass": true,
    "test_pass": true,
    "output_hash": "abc123..."
  },
  "skip_conditions": {
    "requires": ["ANTHROPIC_API_KEY"],
    "skip_message": "SKIP-OK: #ticket"
  }
}
```

The 5 Validation Layers

1. Directory structure — files exist in expected locations
2. Code quality — lint checks pass
3. Compilation — project builds
4. Testing — generated tests pass
5. Runtime validation — output matches expected

Anti-Bluff Requirements

- Every assertion must produce positive runtime evidence

- No absence-of-error passes
- No metadata-only assertions
- No grep-based PASS without execution
- Every t.Skip() must have SKIP-OK: #

Provider Gating

```
if apiKey := os.Getenv("ANTHROPIC_API_KEY"); apiKey == "" {
    fmt.Println("SKIP-OK: #constitution-gate ANTHROPIC_API_KEY
        not set")
    os.Exit(0) // pass, not fail
}
```

Checklist

- [ ] challenge.go is self-contained runnable
- [ ] expected.json has evidence assertions
- [ ] All assertions produce real output
- [ ] SKIP-OK markers for gated tests
- [ ] Exit 0 = PASS, non-zero = FAIL

```
- [ ] **Commit**

---

### Task P5-T07: Inline Go Documentation

- [ ] **Step 1: Audit current doc comment coverage**

```bash
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
go doc -all ./... 2>&1 | grep "no Go files\|undefined\|no exported" | wc -
```

- [ ] **Step 2: Add doc comments to all exported symbols**

For every exported type, function, method, constant, variable in: - `internal/` packages - `cmd/` entry points - `applications/` platform code

Format: `// Name does X. ...`

- [ ] **Step 3: Verify coverage**
```

```
cd HelixCode
# Count exported symbols vs documented
go doc -all ./internal/... 2>&1 | grep -c "func \|type \|var \|
        const "
```

All must have doc comments.

- [ ] **Step 4: Commit batch-by-batch (one package at a time)**

```
for pkg in internal/auth internal/llm internal/server internal/
        tools internal/worker internal/task internal/memory
        internal/config internal/verifier; do
  cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
  git add "helix_code/$pkg/"
  git commit -m "docs(P5-T07): add Go doc comments to $pkg

Phase: 5  Task: P5-T07"
done
```

Task P5-T08: Update GAP_ANALYSIS.md

Files: Modify `GAP_ANALYSIS.md`

- [ ] **Mark all Phase 3 features as COMPLETE with evidence links:**

```
### ✅ LiteLLM Abstraction Layer (COMPLETED — P3-T01)
- File: internal/llm/litellm/
- Evidence: go test ./internal/llm/litellm/ PASS

### ✅ RepoMap (COMPLETED — P3-T02)
- File: internal/repomap/
- Evidence: go test ./internal/repomap/ PASS

### ✅ Quality Scoring (COMPLETED — P3-T03)
- File: internal/quality/
- Evidence: go test ./internal/quality/ PASS

### ✅ Interactive Clarification (COMPLETED — P3-T04)
- File: internal/clarification/
- Evidence: go test ./internal/clarification/ PASS

### ✅ Plugin System (COMPLETED — P3-T05)
- File: internal/plugins/
- Evidence: go test ./internal/plugins/ PASS

### ✅ Cohere Provider (COMPLETED — P3-T06)
```

✅ Replicate Provider (COMPLETED — P3-T07)
✅ Together.ai Provider (COMPLETED — P3-T08)

- [ ] **Update provider count from 12 to 15**
- [ ] **Commit**

Task P5-T09/P5-T10/P5-T11/P5-T12/P5-T13: Update all remaining docs

- [ ] **P5-T09**: Update `HELIXCODE_FEATURE_GAP_ANALYSIS.md` (mirror T08)
- [ ] **P5-T10**: Update `PROGRESS.md` — document Phase 1-5 completion, close all tasks, set "Active phase" to none
- [ ] **P5-T11**: Update `CONTINUATION.md` — set current state to "Phases 1-5 complete", list any deferred items
- [ ] **P5-T12**: Update `AGENTS.md` — mark all BLUFF/STUB as FIXED, add verified real features, remove old references
- [ ] **P5-T13**: Update `HELIXCODE_GAP_ANALYSIS.md` — sync with T08/T09
- [ ] **Commit as one batch:**

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add GAP_ANALYSIS.md HELIXCODE_FEATURE_GAP_ANALYSIS.md
        HELIXCODE_GAP_ANALYSIS.md \
        AGENTS.md docs/improvements/PROGRESS.md docs/
        CONTINUATION.md
git commit -m "docs(P5-T08-T13): update all existing materials
        for Phase 1-5 completion

Phase: 5  Tasks: P5-T08 through P5-T13"
```

Task P5-T14: Documentation completeness check

- [ ] **Step 1: Verify every documented feature has a working challenge**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
# For each guide, verify referenced features exist in tree
```

```
for guide in docs/guides/*.md; do
  echo "=== $guide ==="
  grep -c "✅\|COMPLETE\|PASS" "$guide" 2>/dev/null || echo "no
      markers"
done
```

- [ ] **Step 2: Verify no stale references**

```
grep -rn "\.bak\|\.old\|Example_Projects" docs/ && echo "STALE
      REFERENCES" || echo "clean"
```

Expected: `clean`

- [ ] **Step 3: Commit**

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add docs/
git commit -m "docs(P5-T14): documentation completeness
      verification

Phase: 5  Task: P5-T14"
```

Task P5-T15: Final anti-bluff sweep

- [ ] **Step 1: Complete anti-bluff grep across all source**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
echo "=== Simulated ===" && grep -rn "simulated\|Simulated"
      internal/ cmd/ applications/ --include="*.go" | grep -v
      "_test.go" | grep -v "doc.go" | grep -v "faiss_fallback"
      || echo "PASS"
echo "=== Placeholder ===" && grep -rn "placeholder\|
      Placeholder" internal/ cmd/ --include="*.go" | grep -v
      "_test.go" | grep -v "doc.go" || echo "PASS"
echo "=== Stub ===" && grep -rn "stub\|Stub" internal/ cmd/ --
      include="*.go" | grep -v "_test.go" | grep -v "doc.go"
      || echo "PASS"
echo "=== TODO ===" && grep -rn "TODO" internal/ cmd/ --
      include="*.go" | grep -v "_test.go" | grep -v "doc.go"
      || echo "PASS"
echo "=== FIXME ===" && grep -rn "FIXME" internal/ cmd/ --
      include="*.go" | grep -v "_test.go" || echo "PASS"
```

Expected: PASS (clean) for all five

- [ ]

Step 2: Skip markers audit

```
grep -rn 't\.Skip(' --include="*_test.go" . | grep -v "SKIP-OK"
        | grep -v "//.*SKIP" && echo "MISSING MARKERS" || echo
        "PASS: all have SKIP-OK"
```

Expected: `PASS: all have SKIP-OK`

- [ ] **Step 3: Build + test + vet**

```
cd HelixCode && go build ./... && go vet ./... && go test
        -short ./...
```

All must pass.

- [ ] **Commit**

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git add -A
git commit -m "chore(P5-T15): final anti-bluff sweep — ALL CLEAN

Phase: 5  Task: P5-T15  Evidence: zero hits on all 5 patterns"
```

Task P5-T16: Final push to all remotes

- [ ] **Step 1: Verify clean status**

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git status  # must be clean
git -C HelixAgent status  # must be clean
git -C helix_qa status  # must be clean
git -C HelixCode status  # must be clean
```

- [ ] **Step 2: Push to all remotes**

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git push github main && git push gitlab main && git push origin
        main && git push upstream main
```

- [ ] **Step 3: Final governance verification**

```
bash scripts/verify-governance-cascade.sh
```

Expected: `PASS`

- [ ] **Step 4: Run anti-bluff verifier challenge**

```
cd helix_code/tests/e2e/challenges/anti_bluff_verifier && go run
        challenge.go
```

Expected: `ANTI-BLUFF: ALL CLEAN`

- [ ] **Step 5: Phase 5 close-out commit**

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
git commit --allow-empty -m "chore(P5-T16): Phase 5 complete —
        HelixCode Zero-Bluff Completion DONE

Phase: 5  Task: P5-T16
Evidence: final anti-bluff sweep CLEAN, governance cascade PASS,
        all tests PASS"
git push github main
```

Phase 5 Completion Checklist

- [ ] 6 standalone guides written and committed
- [ ] Go doc comments on all exported symbols
- [ ] GAP_ANALYSIS, PROGRESS, CONTINUATION, AGENTS updated
- [ ] Documentation completeness verified
- [ ] Final anti-bluff sweep: zero hits on all patterns
- [ ] All skip markers present
- [ ] `go build ./...` + `go vet ./...` + `go test -short ./...` all pass
- [ ] Anti-bluff verifier challenge: ALL CLEAN
- [ ] Governance cascade: PASS
- [ ] Pushed to all 4 remotes
- [ ] git status clean across all repos

PROGRAMME COMPLETE.